CS-338 Intro to the Theory of Computation

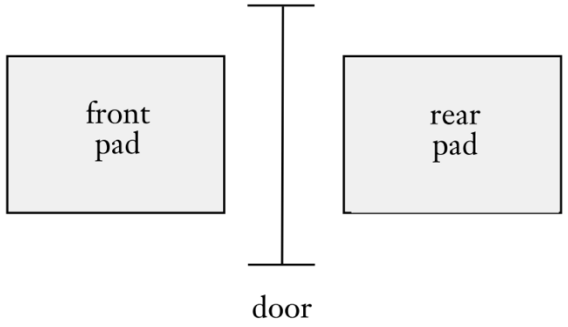
- Lecture #1

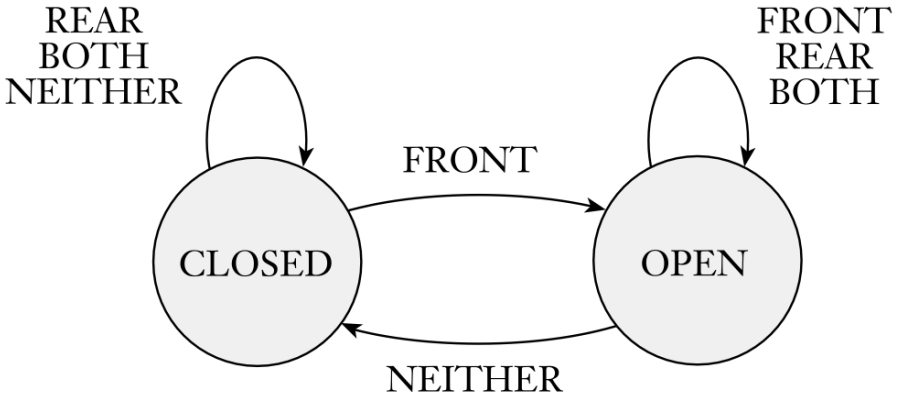
➢ **Deterministic Finite Automata**

- Prof Pietro S. Oliveto
  Department of Computer Science and Engineering
  Southern University of Science and Technology (SUSTech)
- olivetop@sustech.edu.cn
  https://faculty.sustech.edu.cn/olivetop
- Office: Room 113

An Automatic Doorway Controller

- What is a computer?
- We interact with computers all the time…



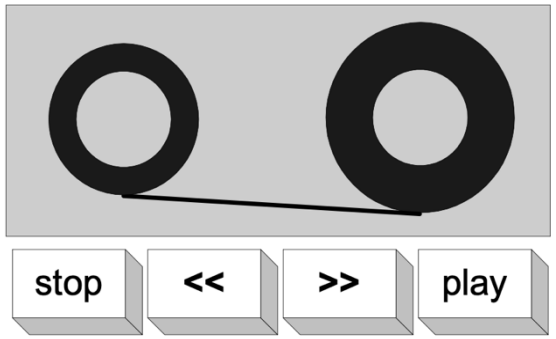


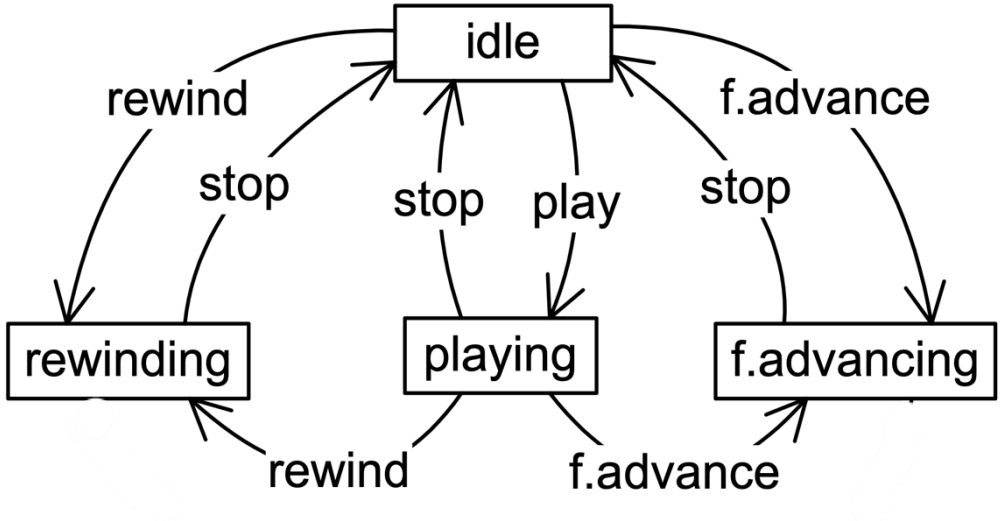
input signal

	NEITHER	FRONT	REAR	BOTH
CLOSED	CLOSED	OPEN	CLOSED	CLOSED
OPEN	CLOSED	OPEN	OPEN	OPEN

This controller is a computer that just has a single bit of memory, capable of recording which of two states the controller is in

A Tape Player Controller





This controller is a computer that has two bits of memory, capable of recording which of four states the controller is in

Other common devices

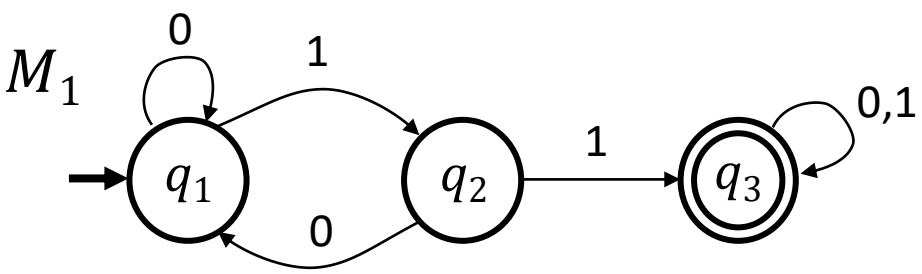
- Other common devices have controllers with somewhat larger memories
- An elevator might have states indicating which floor it is on
- Dishwashers, elecronic thermostats also may have many states
- Vending machines etc.

- These are all finite automata
- Their operation depends on the state they currently are in, and the input they receive

What can they compute and what can they not compute?

We first need to define the model of computation precisely…

Let's begin: Finite Automata



States: $q_1\ q_2\ q_3$

Transitions: $\xrightarrow{1}$

Start state:

Accept states:

Input: finite string
Output: Accept or Reject

Computation process: Begin at start state, read input symbols, follow corresponding transitions, Accept if end with accept state, Reject if not.

Examples: 01101 → Accept
00101 → Reject

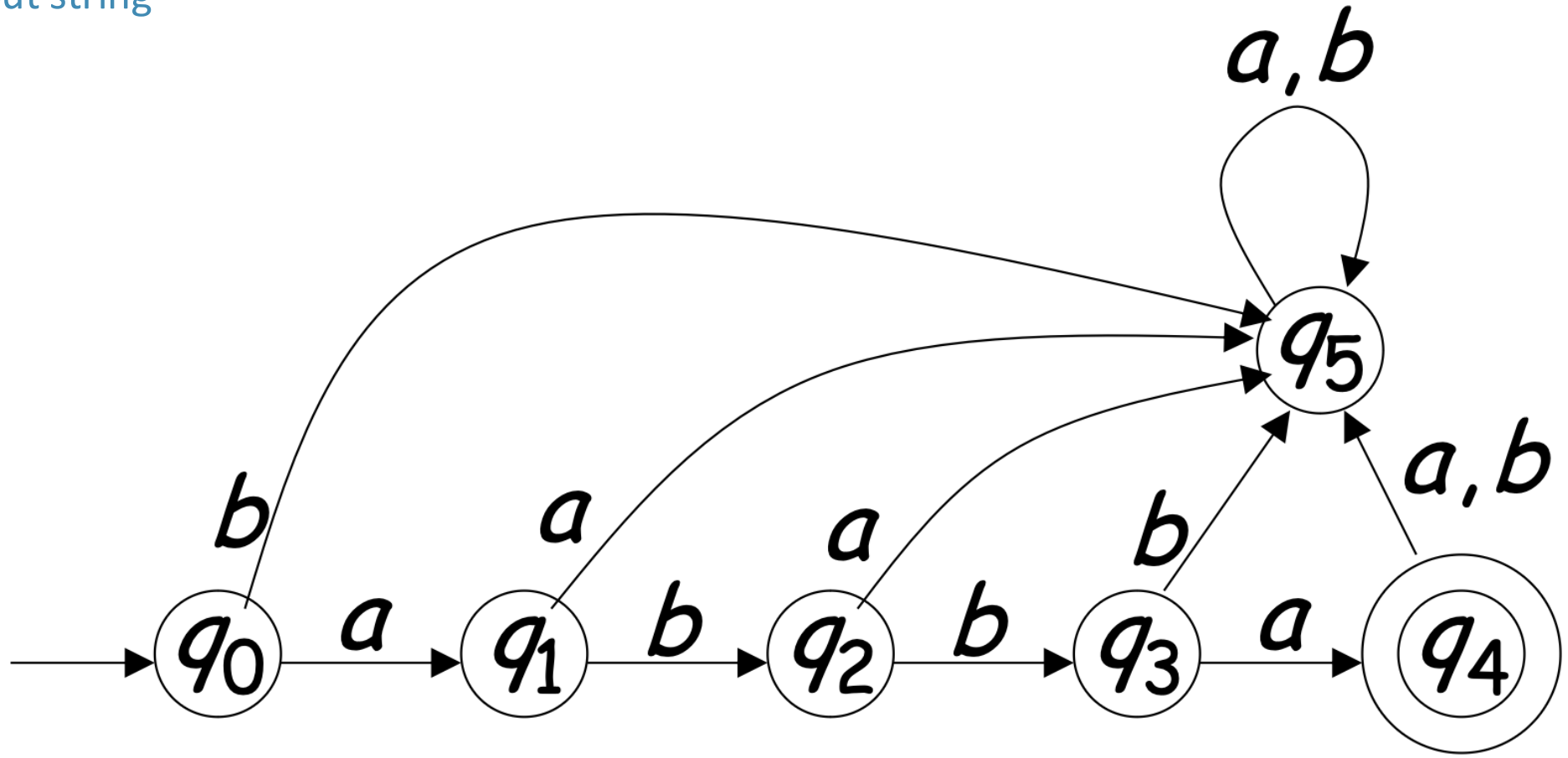
M_1 accepts exactly those strings in A where
$A = \{w \mid w \text{ contains substring } 11\}$.

Say that A is the language of M_1 and that M_1 recognizes A and that $A = L(M_1)$.

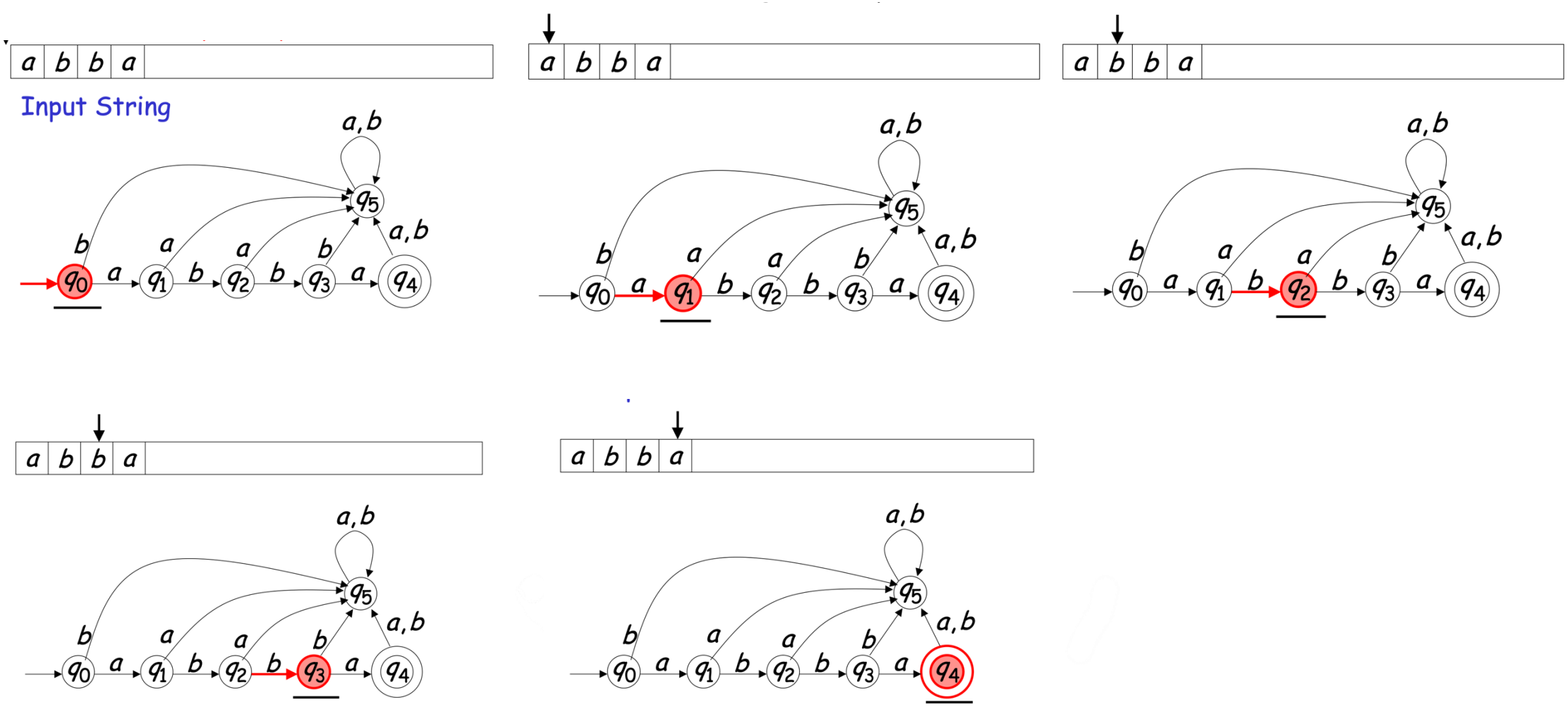
Finite Automata: Example

a	b	b	a

Input string

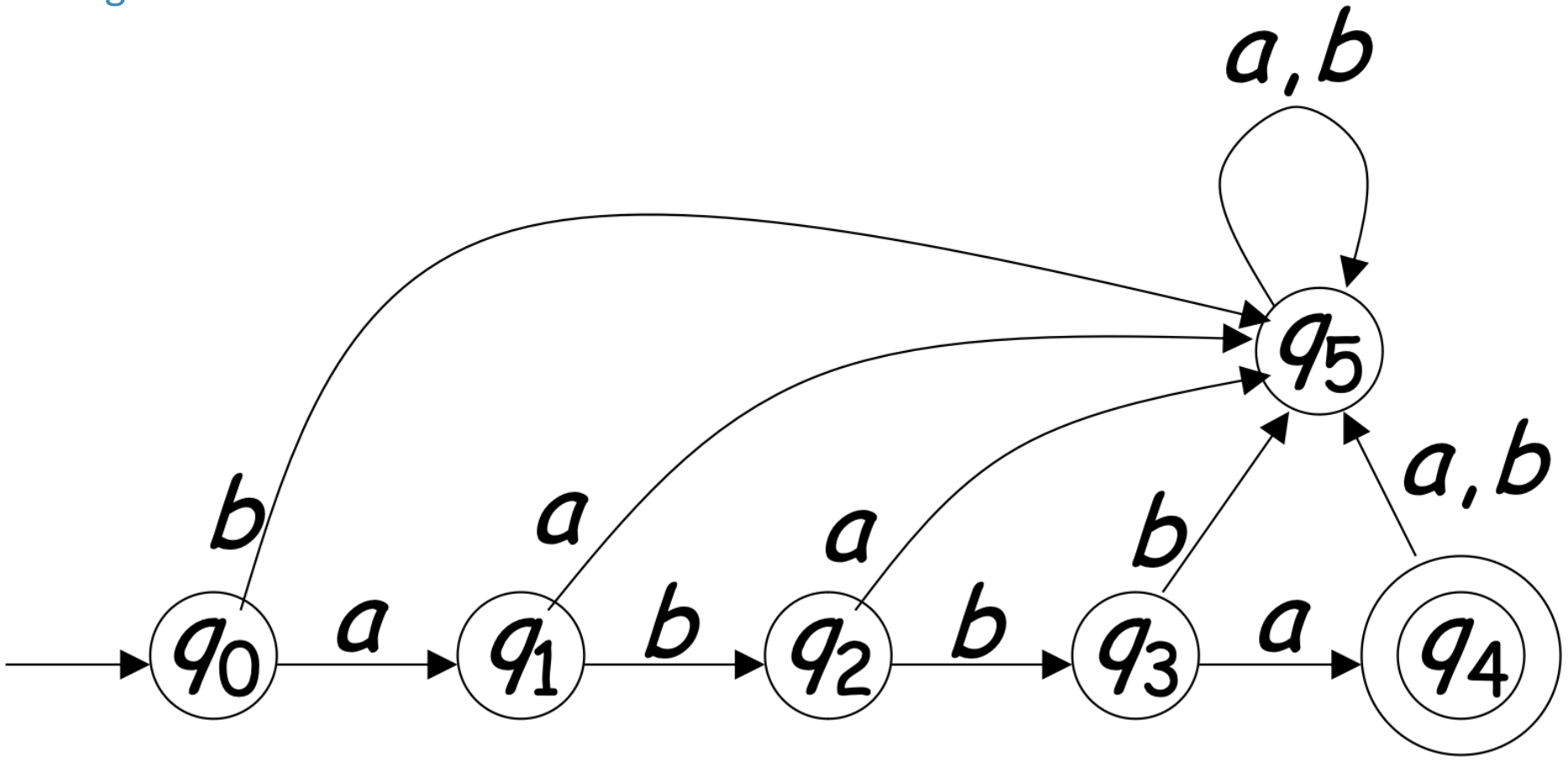


Finite Automata: Example

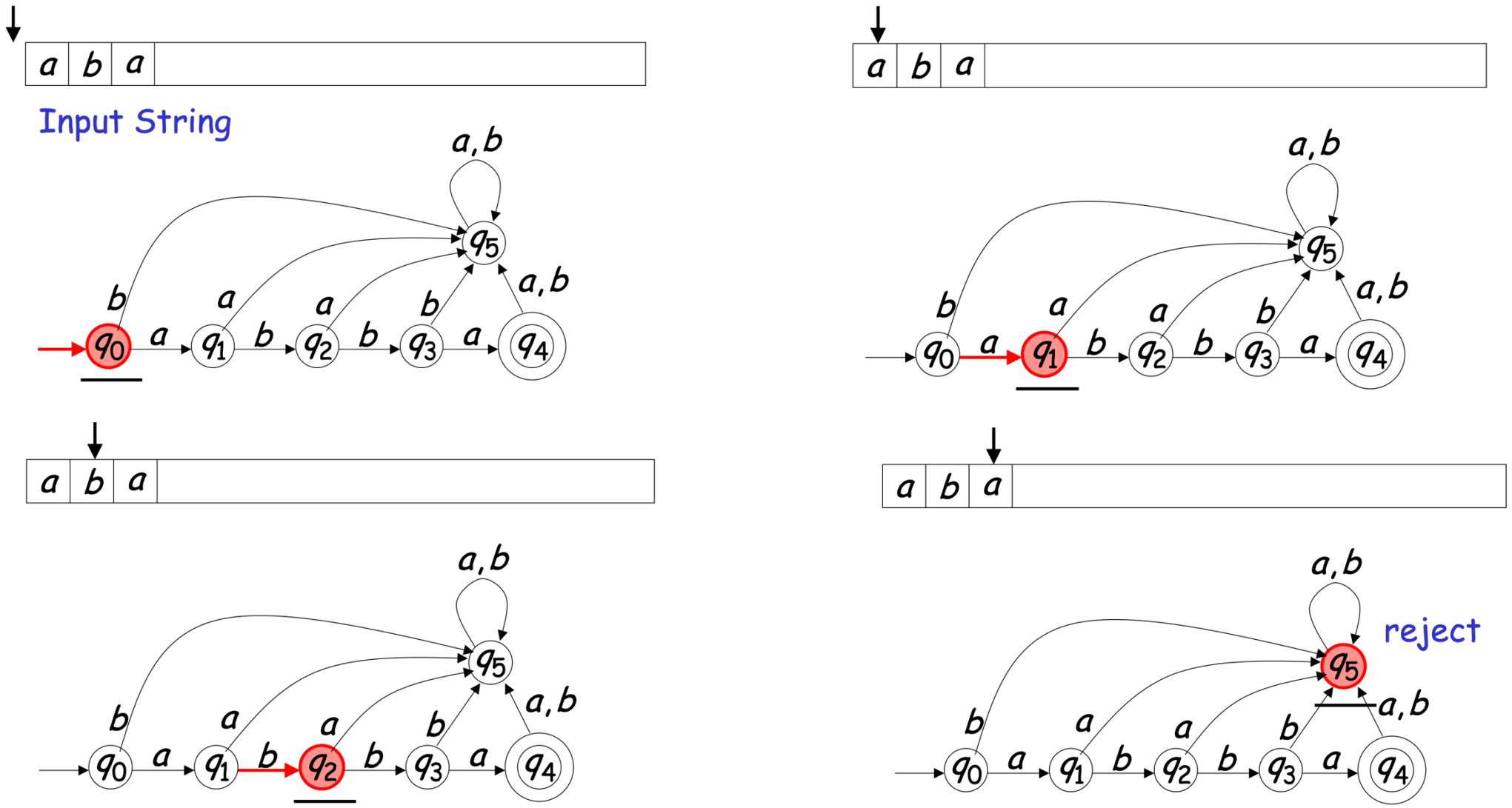


Finite Automata: Example

Input string

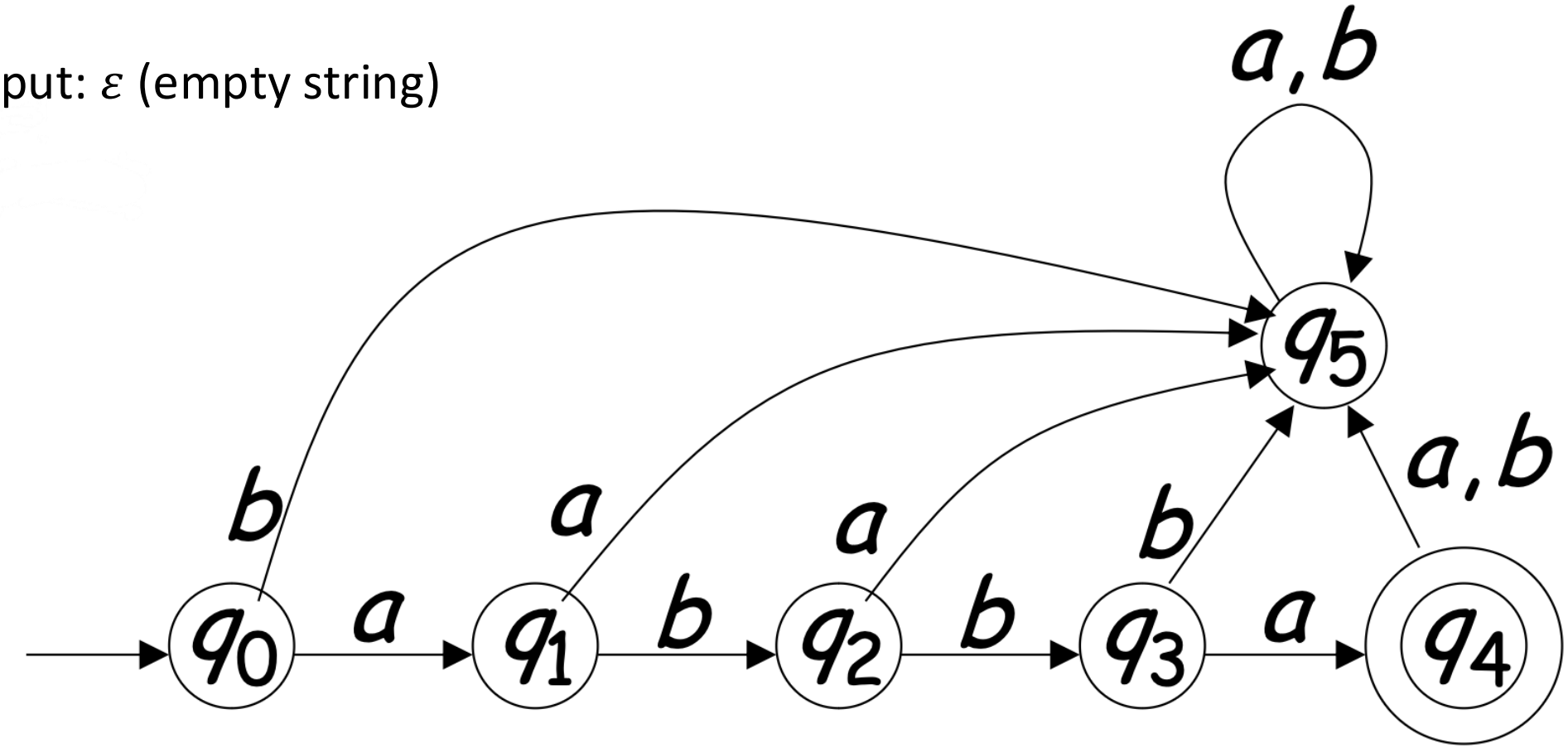


Finite Automata: Example



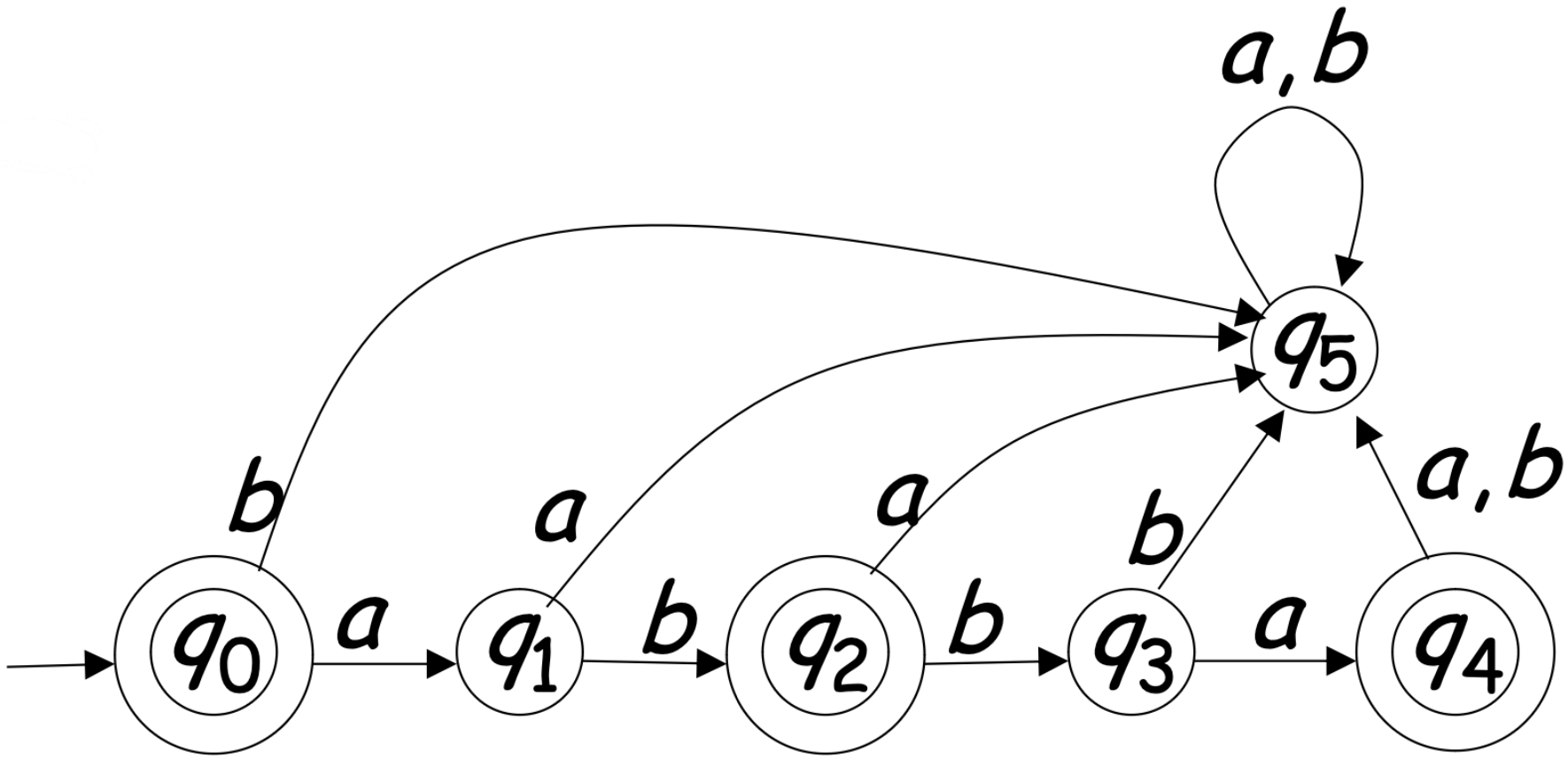
Finite Automata: Example

Input: ε (empty string)



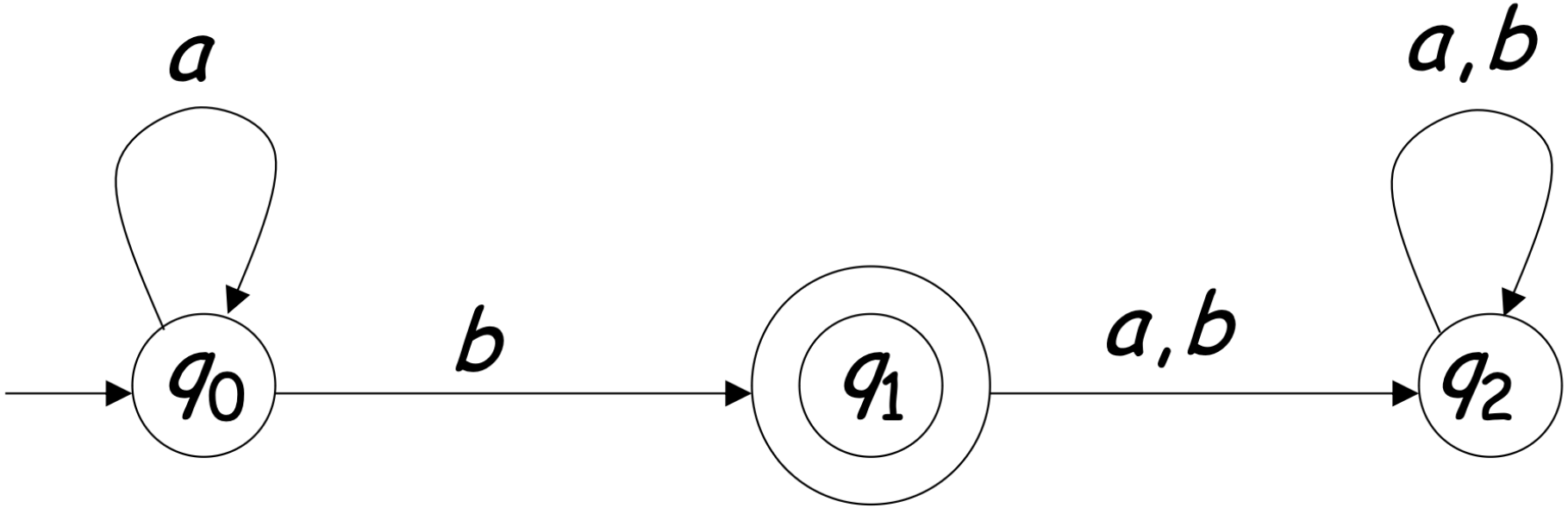
The language accepted by this Finite State Machine is? $L = \{abba\}$

Finite Automata: Another Example (2)

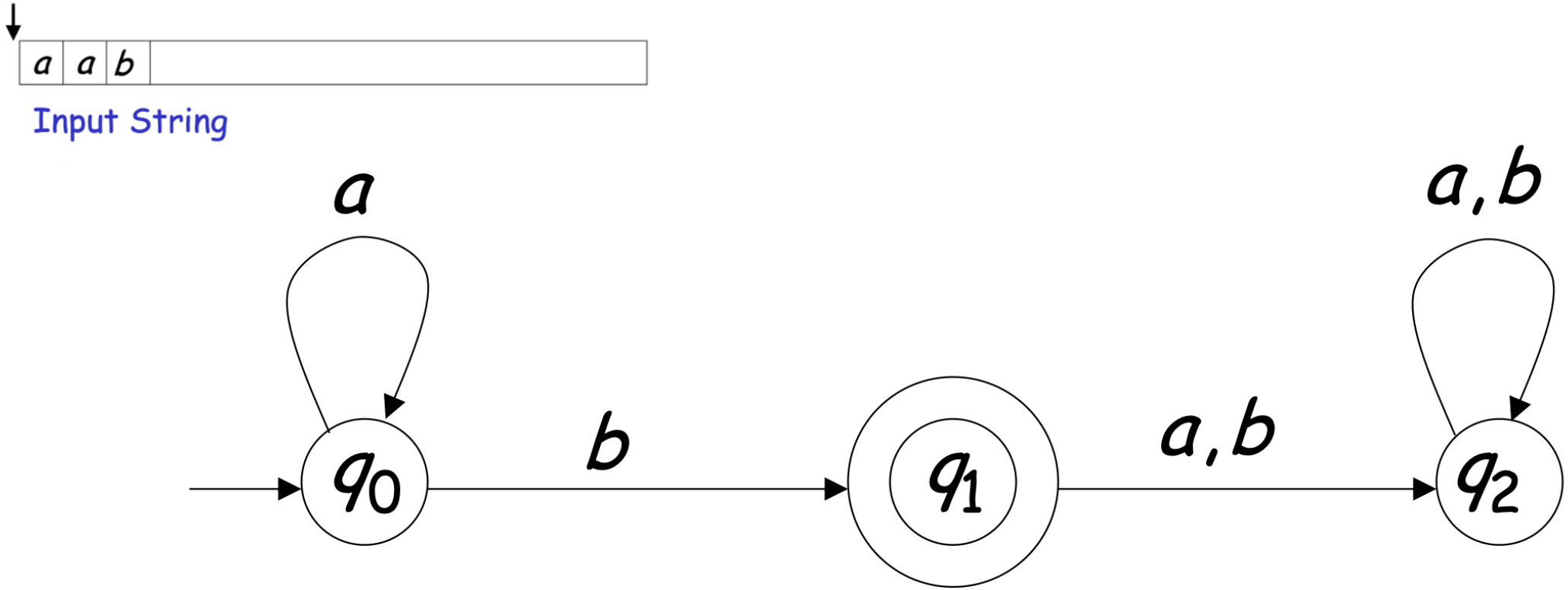


The language accepted by this Finite State Machine is? $L = \{\varepsilon, ab, abba\}$

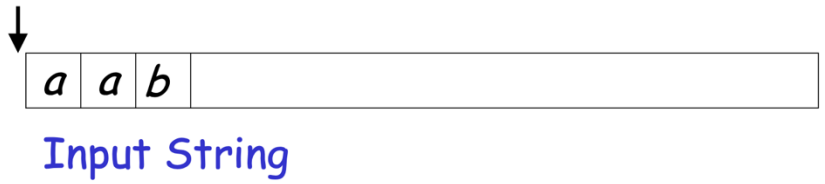
Finite Automata: Another Example (3)

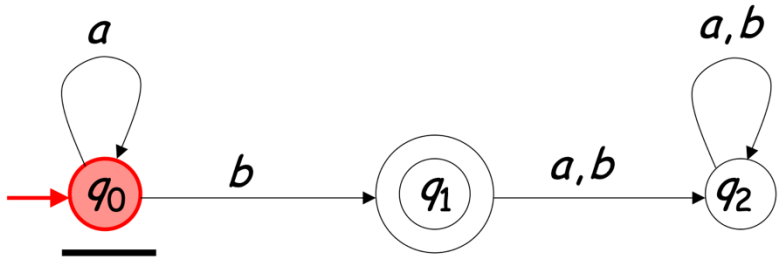


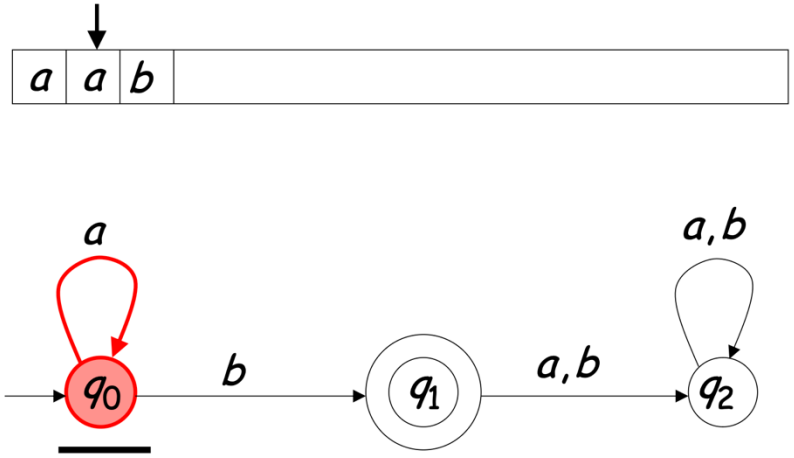
Finite Automata: Another Example (3)

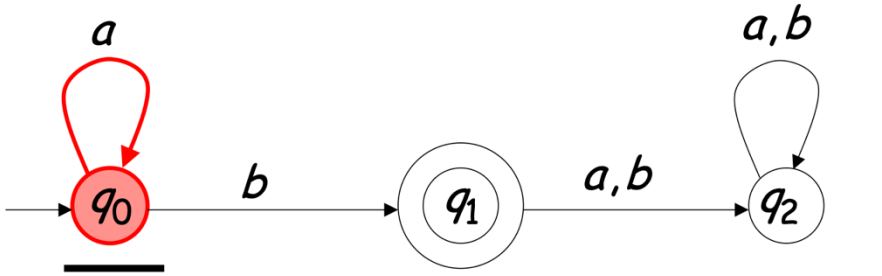


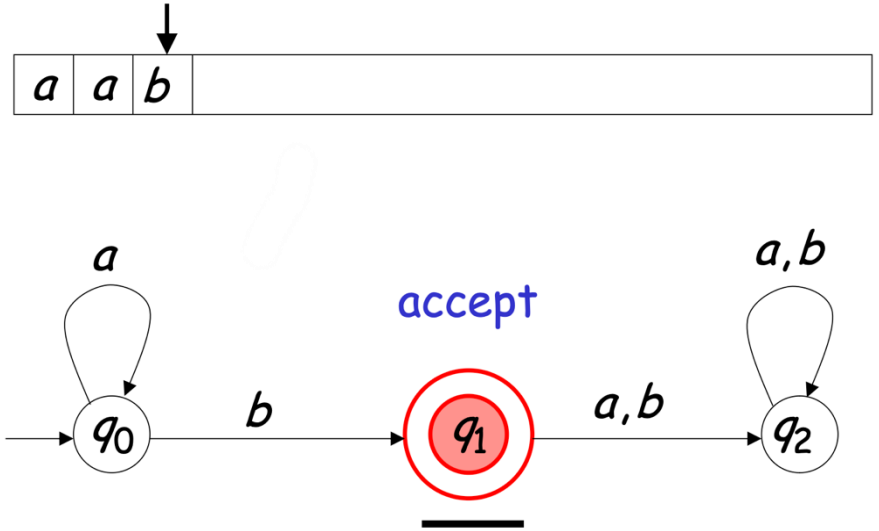
Finite Automata: Another Example (3)



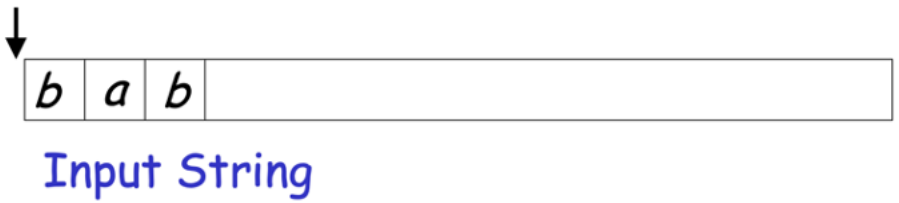


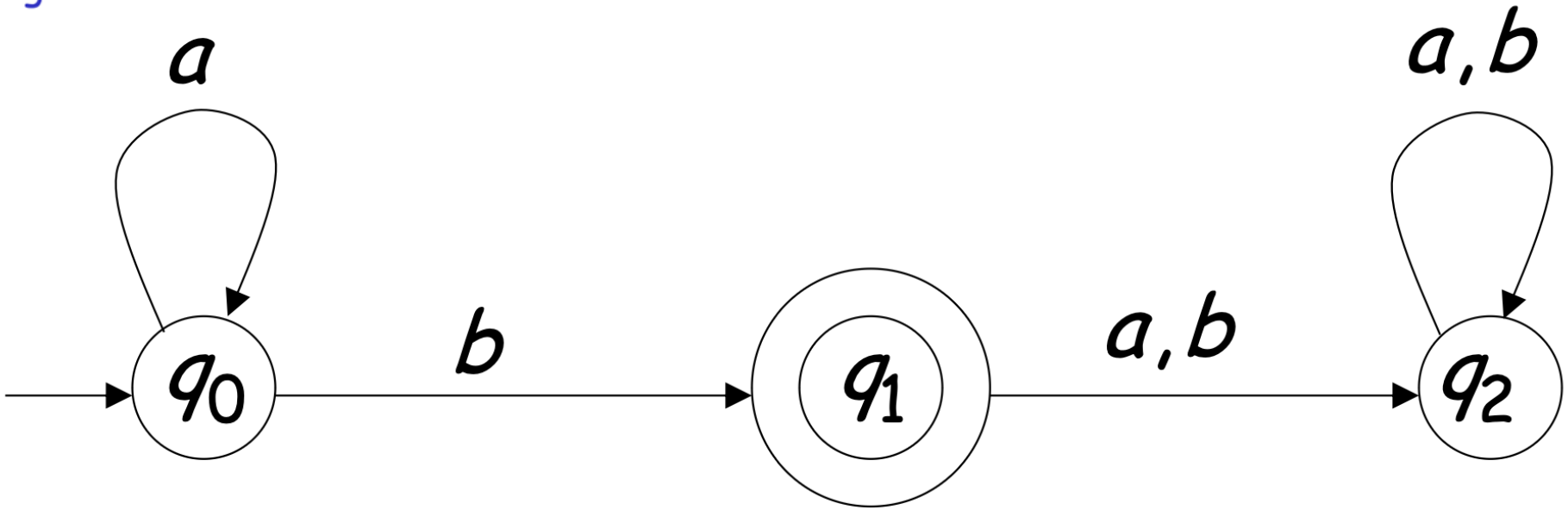




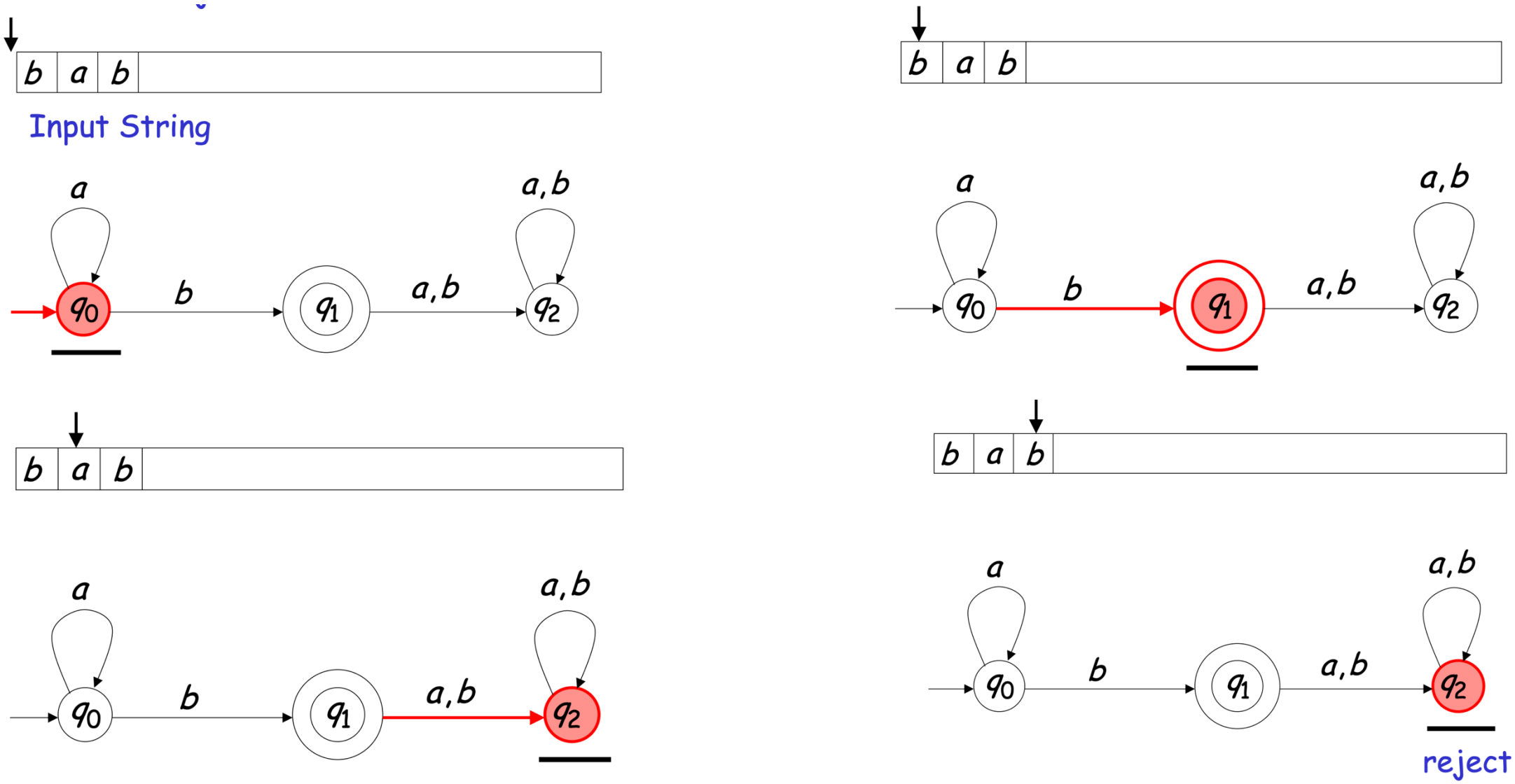


Finite Automata: Another Example (3)

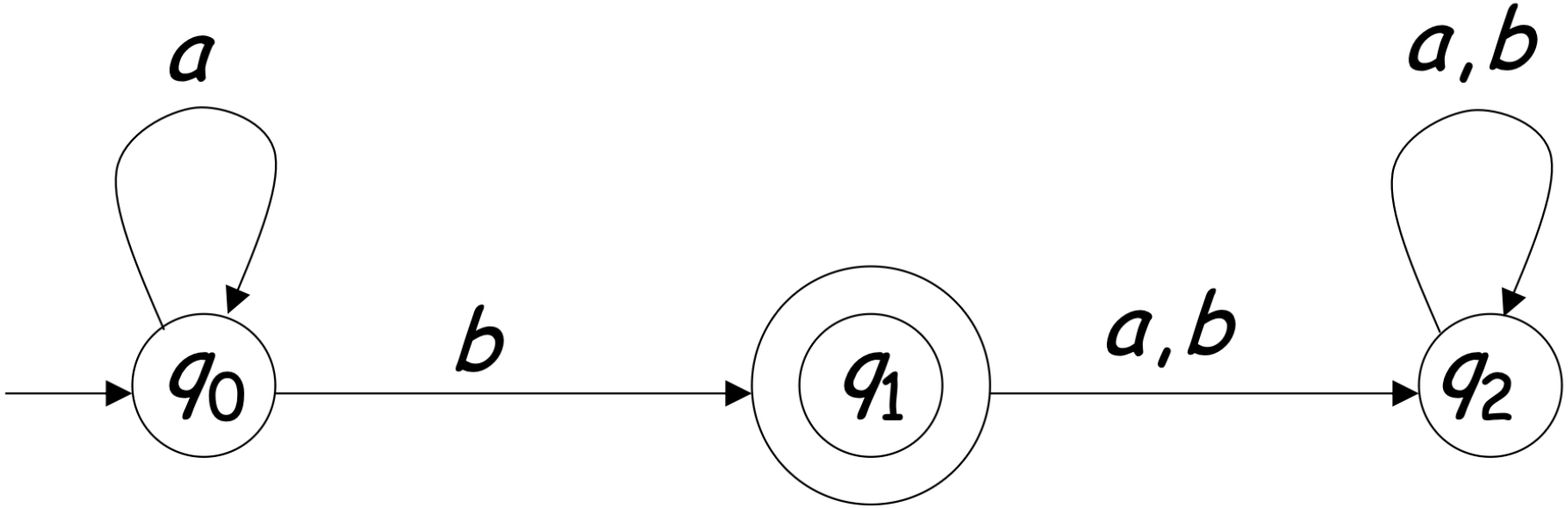




Finite Automata: Another Example (3)



Finite Automata: Another Example (3)



The language accepted by this Finite State Machine is? $L = \{a^n b : n \geq 0\}$

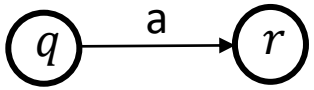
Finite Automata – Formal Definition

Defn: A <u>finite automaton</u> M is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$

Q finite set of states

Σ finite set of alphabet symbols

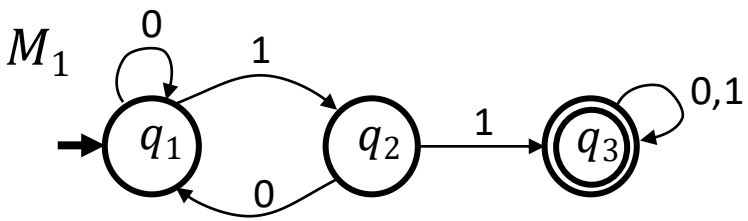
δ transition function $\delta\colon Q \times \Sigma \to Q$

$\delta(q, a) = r$ means 

q_0 start state

F set of accept states

Example:



$M_1 = (Q, \Sigma, \delta, q_1, F)$

$Q = \{q_1, q_2, q_3\}$

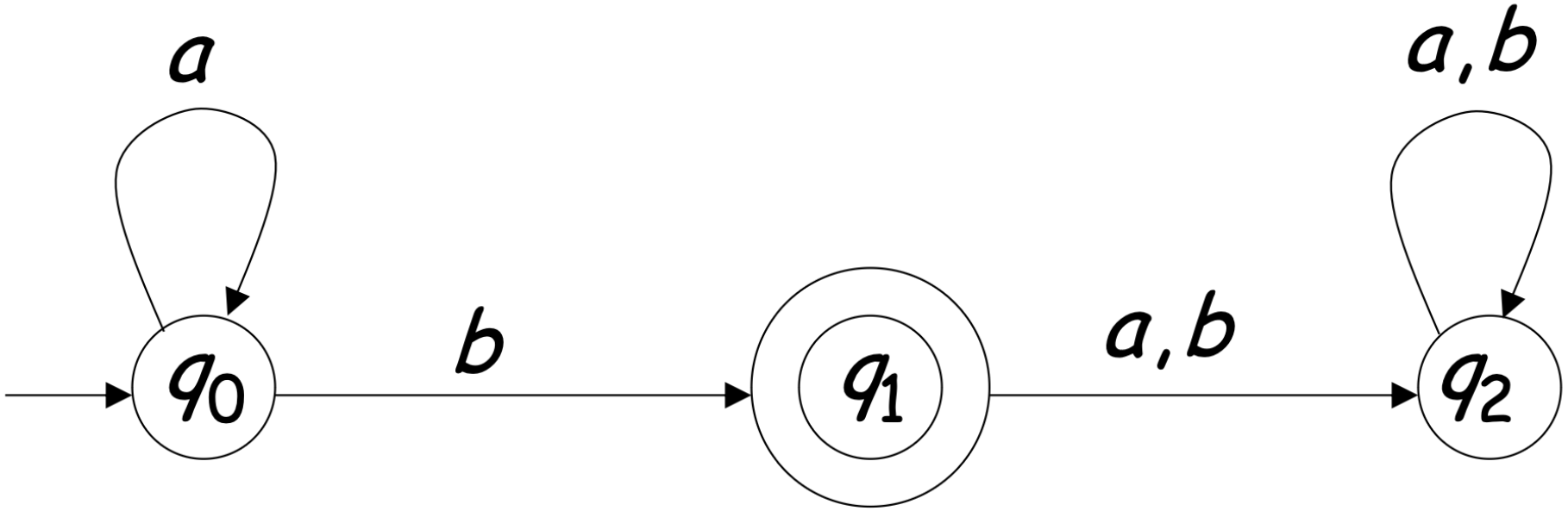
$\Sigma = \{0, 1\}$

$F = \{q_3\}$

$\delta =$

	0	1
q_1	q_1	q_2
q_2	q_1	q_3
q_3	q_3	q_3

Finite Automata Definition: Example (3) Revisited



$M_1 = (Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{a, b\}$

$F = \{q_1\}$

$\delta =$

	a	b
q_0	q_0	q_1
q_1	q_2	q_2
q_2	q_2	q_2

The language accepted by this Finite State Machine is? $L = \{a^n b : n \geq 0\}$

Finite Automata Definition: Example (2) revisited

$M_1 = (Q, \Sigma, \delta, q_0, F)$

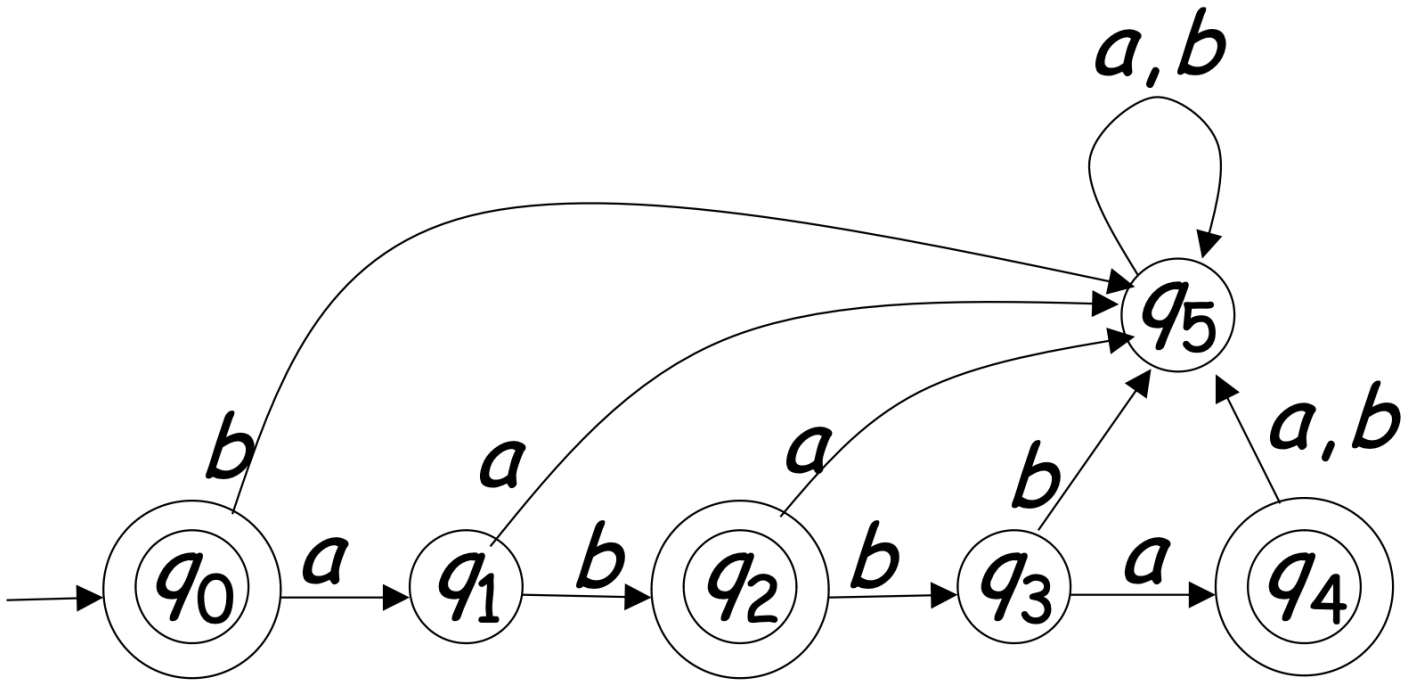
$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$

$\Sigma = \{a, b\}$

$F = \{q_0, q_2, q_4\}$

$\delta =$

	a	b
q_0	q_1	q_5
q_1	q_5	q_2
q_2	q_5	q_3
q_3	q_4	q_5
q_4	q_5	q_5
q_5	q_5	q_5



The language accepted by this Finite State Machine is?

$$L = \{\varepsilon, ab, abba\}$$

Finite Automata – Computation

Strings and languages

- A <u>string</u> is a finite sequence of symbols in Σ
- A <u>language</u> is a set of strings (finite or infinite)
- The <u>empty string</u> ε is the string of length 0
- The <u>empty language</u> $\emptyset$ is the set with no strings

Let $M = (Q, \Sigma, \delta, q_0, F)$
and let $w = w_1 w_2 \ldots w_n$ with each $w_i \in \Sigma$

Defn: M <u>accepts string</u> $w = w_1 w_2 \ldots w_n$ each $w_i \in \Sigma$
if there is a sequence of states $r_0, r_1, r_2, , \ldots, r_n \in Q$
where:

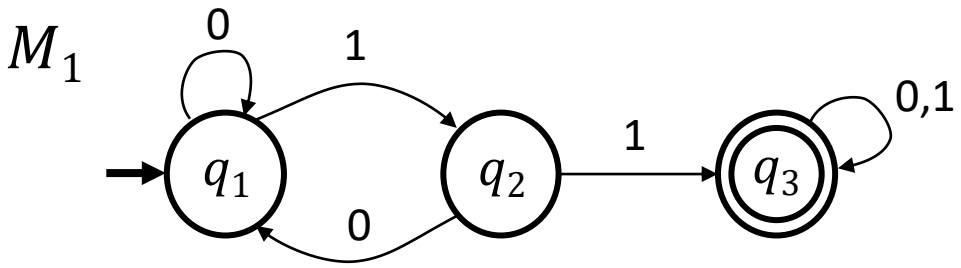
1. $r_0 = q_0$
2. $r_i = \delta(r_{i-1}, w_i)$ for $1 \leq i \leq n$
3. $r_n \in F$

Recognizing languages

- $L(M) = \{w \mid M \text{ accepts } w\}$
- $L(M)$ is <u>the language of</u> M
- M <u>recognizes</u> $L(M)$

Defn: A language is <u>regular</u> if some finite automaton recognizes it.

Regular Languages – Examples



$L(M_1) = \{w | \, w \text{ contains substring } 11\} = A$

<u>Therefore A is regular</u>

More examples:

Let $B = \{w | \, w$ has an even number of 1s$\}$
B is regular (make automaton for practice).

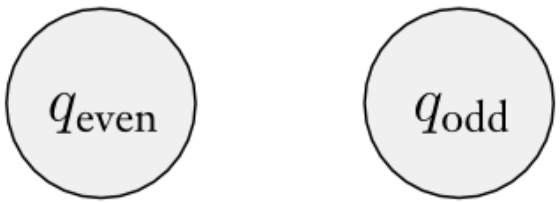
Let $C = \{w | \, w$ has equal numbers of 0s and 1s$\}$
C is <u>not</u> regular (we will prove).

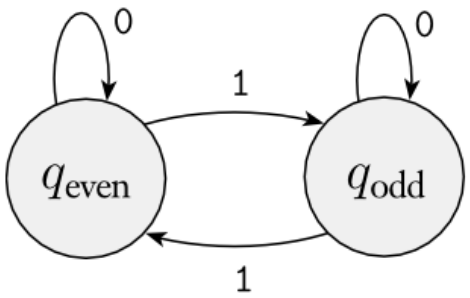
Goal: Understand the regular languages
(what are the languages that are recognized by a finite automaton).

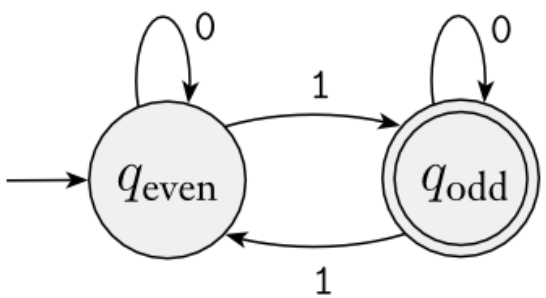
Regular language: Example

Let $D = \{w |\ w$ has an odd number of 1s$\}$
Show that D is regular.

Let's assume the alphabet is $\Sigma = \{0,1\}$



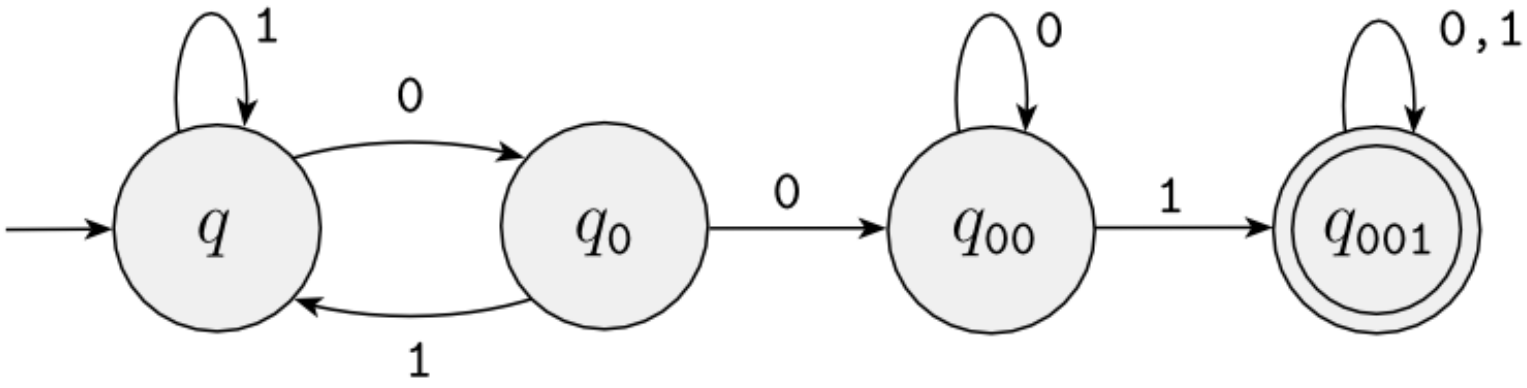




Regular language: Example (2)

Let $E = \{w | \, w$ contains 001 as a substring$\}$
Show that E is regular.

Let's assume the alphabet is $\Sigma = \{0,1\}$



Operations on languages: Regular Expressions

- In arithmetic the basic objects are numbers and operations such as +,*,-,/ to manipulate them
- In Theory of Computation, the objects are languages: we define three important operations:

Regular operations. Let A, B be languages:

- Union: $A \cup B = \{w \mid w \in A \text{ or } w \in B\}$
- Concatenation: $A \circ B = \{xy \mid x \in A \text{ and } y \in B\} = AB$
- Star: $A^* = \{x_1 \dots x_k \mid \text{each } x_i \in A \text{ for } k \geq 0\}$

  Note: $\varepsilon \in A^*$ always

Example. Let $A = \{\text{good, bad}\}$ and $B = \{\text{boy, girl}\}$.

- $A \cup B = \{\text{good, bad, boy, girl}\}$
- $A \circ B = AB = \{\text{goodboy, goodgirl, badboy, badgirl}\}$
- $A^* = \{\varepsilon, \text{good, bad, goodgood, goodbad, badgood, badbad, goodgoodgood, goodgoodbad, } \dots\}$

Regular expressions

- Built from Σ, members $\Sigma, \emptyset, \varepsilon$ [Atomic]
- By using $\cup, \circ, *$ [Composite]

Examples:

- $(0 \cup 1)^* = \Sigma^*$ gives all strings over Σ={0,1}
- $\Sigma^* 1$ gives all strings that end with 1
- $\Sigma^* 11 \Sigma^*$ = all strings that contain 11 $= L(M_1)$

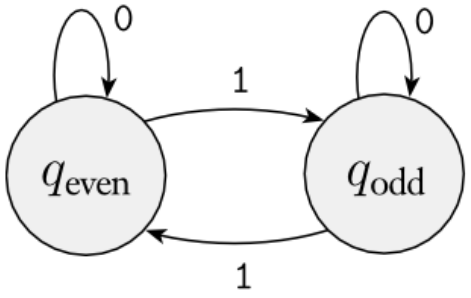
Goal: Show finite automata equivalent to regular expressions

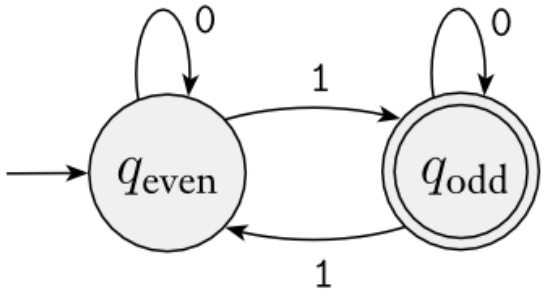
(any regular expression can be converted into a finite automaton that recognises the language it describes, and viceversa)

Regular language: Example

Let $D = \{w \mid w \text{ has an odd number of 1s}\}$

$\Sigma = \{0,1\}$



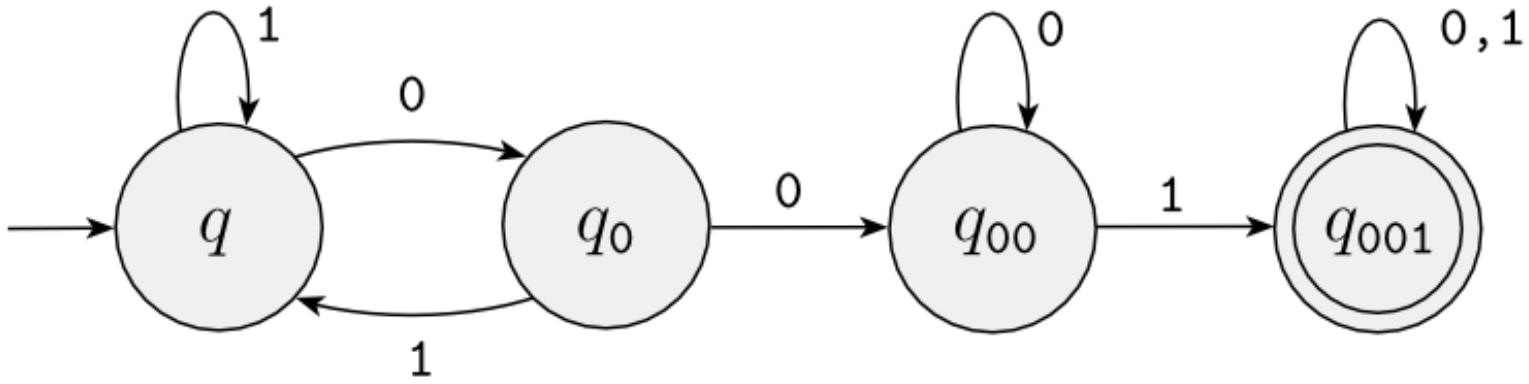


Regular expression? $0^*10^*(0^*10^*10^*)^*$ [or equivalently $0^* \circ 1 \circ 0^* \circ (0^* \circ 1 \circ 0^* \circ 1 \circ 0^*)^*$]

Regular language: Example (2)

Let $E = \{w \mid w \text{ contains 001 as a substring}\}$

$\Sigma = \{0,1\}$



Regular expression? $\Sigma^* 001 \Sigma^*$ [or equivalently $\Sigma^* \circ (0 \circ 0 \circ 1) \circ \Sigma^*$]

Goal: we want to show that finite automata are equivalent to regular expressions

Closure Properties for Regular Languages

A set of objects is **closed** under some operation if applying it to the objects we get an element that belongs to the set

Example

Let $\mathbb{N} = \{1,2,3 \ldots\}$ be the set of natural numbers;

$\mathbb{N}$ is closed under multiplication because any two number x and y we multiply we get $x * y \in \mathbb{N}$

$\mathbb{N}$ is not closed under division as eg., 1 and 2 are in $\mathbb{N}$ but $1/2 \notin \mathbb{N}$

Are regular languages closed under the regular operations?

Eg. Are they closed under union?
(Is the union of two regular languages still a regular language? -> A,B Regular => $A \cup B$ Regular) ?

Next: We show that regular languages are closed under regular operations

Closure Properties: Union

Theorem: If A_1, A_2 are regular languages, so is $A_1 \cup A_2$ (closure under $\cup$)

How to Prove: Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1
Let $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ recognize A_2

Construct $M = (Q, \Sigma, \delta, q_0, F)$ that recognizes $A_1 \cup A_2$

M should accept input w if either M_1 or M_2 accept w.

Proof Idea

- M must accept exactly when either M_1 or M_2 accept w.
- What would you do if you were the machine?
- We construct M by simulating both M_1 and M_2 simultaneously and accepting if either accepts
- However, we can't simulate one machine at a time because we can't "rewind the tape"
- We need to track both machines at the same time by remembering the state each machine would be in at each point! =>
- We need to remember a pair of states and the transition function will go between pairs

Closure Properties: Union

Theorem: If A_1, A_2 are regular languages, so is $A_1 \cup A_2$ (closure under $\cup$)

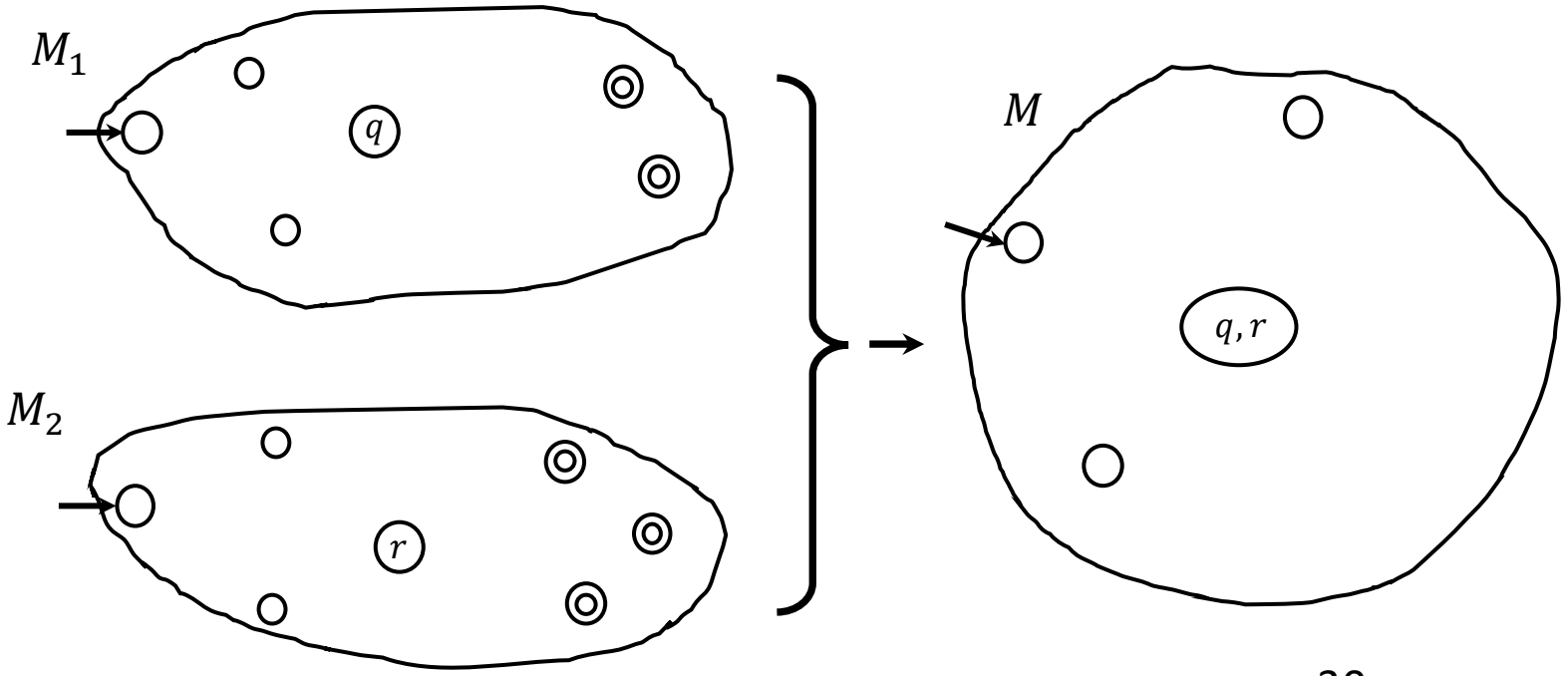
How to prove: Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1

Let $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ recognize A_2

Construct $M = (Q, \Sigma, \delta, q_0, F)$ that recognizes $A_1 \cup A_2$

M should accept input w if either M_1 or M_2 accept w.

Proof



Components of M:

$Q = Q_1 \times Q_2$

$= \{(q_1, q_2) | q_1 \in Q_1 \text{ and } q_2 \in Q_2\}$

$q_0 = (q_1, q_2)$

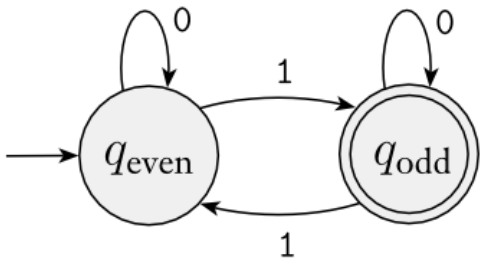
Σ

$\delta((q, r), a) = (\delta_1(q, a), \delta_2(r, a))$

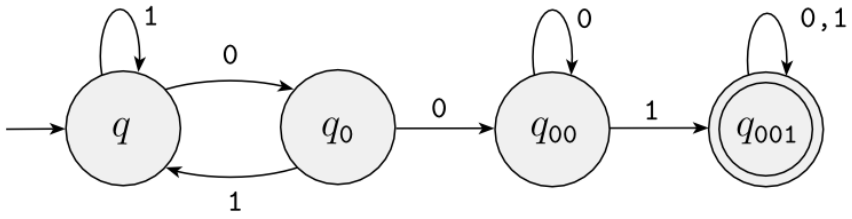
$F = \cancel{F_1 \times F_2}$ NO! [gives intersection]

$F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$

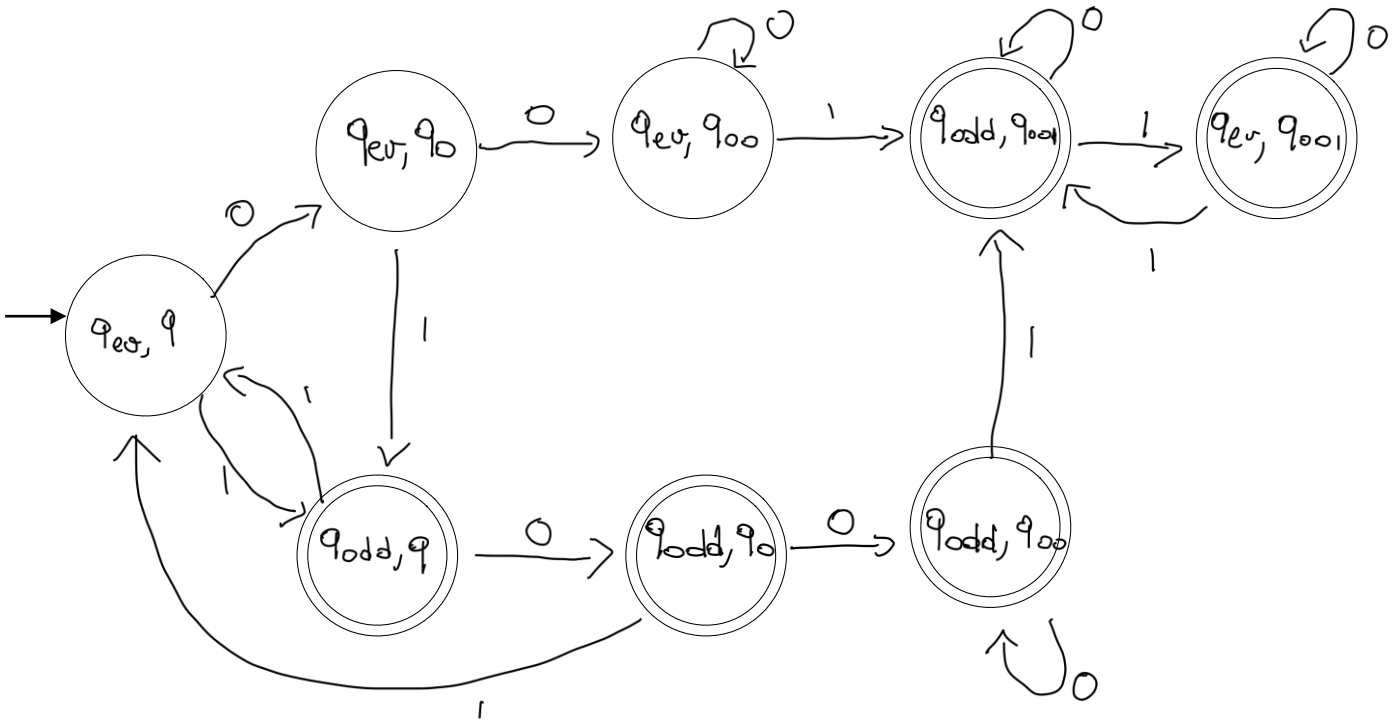
Closure Properties: Union Example



Language $D = \{w|\ w$ has an odd number of 1s$\}$



Language $E = \{w|\ w$ contains 001 as a substring$\}$



Language $D \cup E = \{w|w$ has an odd number of 1s or w contains 001 as a substring$\}$

Components of $\boldsymbol{M}$:

$Q = Q_1 \times Q_2, \quad |Q| = 8$

$q_0 = (q_{ev}, q)$

Σ

$\delta((q,r),a) = (\delta_1(q,a), \delta_2(r,a))$

$F = (F_1 \times Q_2) \cup (Q_1 \times F_2),\ |F| = 5$

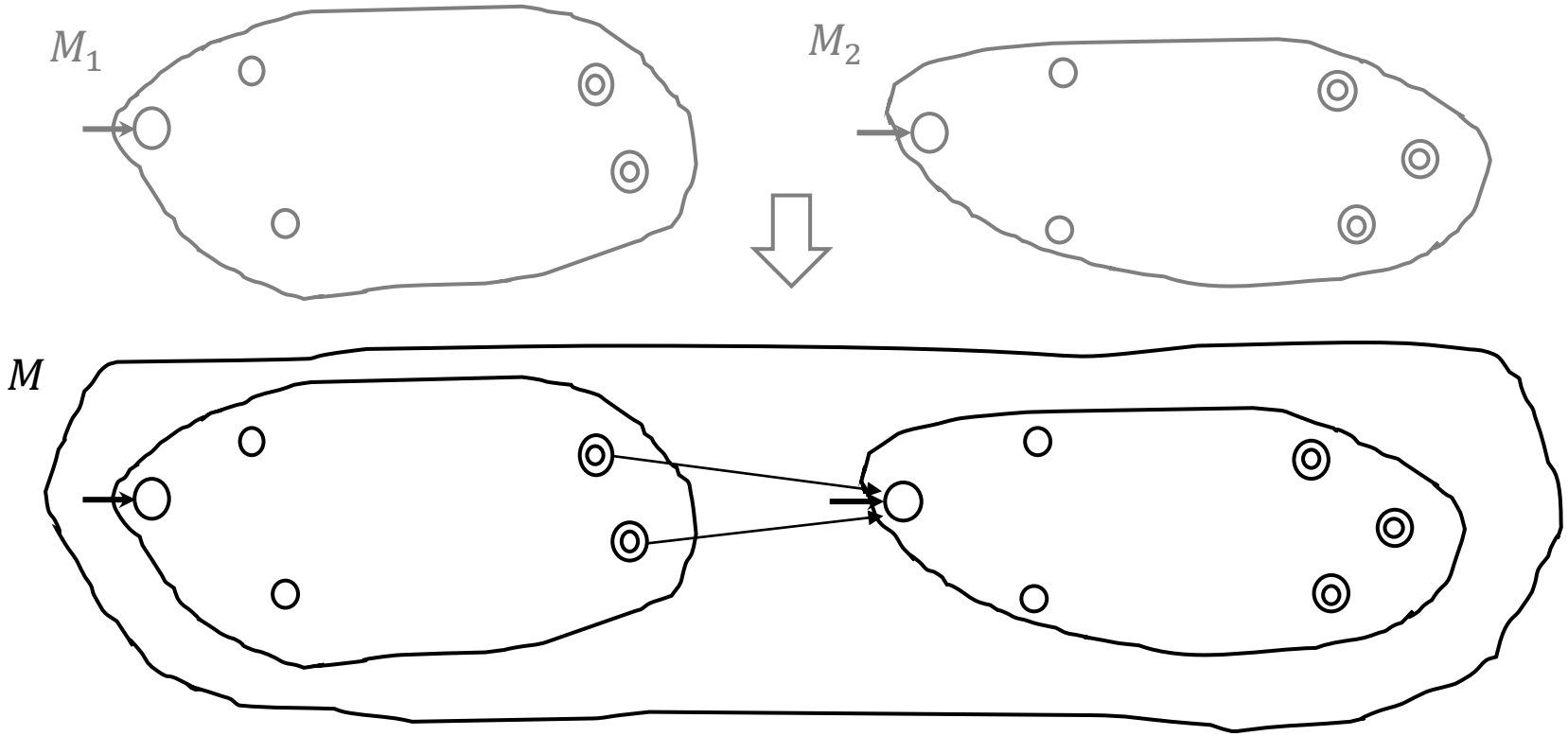
Closure Properties continued

Theorem: If A_1, A_2 are regular languages, so is $A_1 A_2$ (closure under $\circ$)

How to prove: Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1

Let $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ recognize A_2

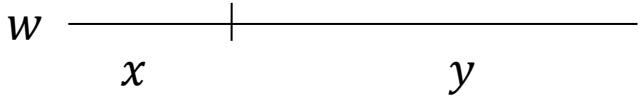
Construct $M = (Q, \Sigma, \delta, q_0, F)$ that recognizes $A_1 \circ A_2$



M should accept input w

if $w = xy$ where

M_1 accepts x and M_2 accepts y.



Doesn't work: Where to split w?

Quick review of today

1. Introduction, outline, mechanics, expectations
2. Finite Automata, formal definition, regular languages
3. Regular Operations and Regular Expressions
4. Proved: Class of regular languages is closed under $\cup$
5. Started: Closure under $\circ$, to be continued…